



UNIVERSIDAD NACIONAL EXPERIMENTAL DE GUAYANA
VICERRECTORADO ACADÉMICO
COORDINACIÓN GENERAL DE PREGRADO
INGENIERÍA INFORMÁTICA
ASIGNATURA: LENGUAJES Y COMPILADORES
SECCIÓN 2

Diferencias en la tokenización entre NLP y lenguajes formales y su implicación con los compiladores



Docente:

Félix Márquez

Integrantes:

Daniel Villalba Santamaría V-27.506.542

Isabella Rodríguez V - 31.521.612

Luis Millán V - 30.040.213

Andrés Gómez V - 31.085.717

Ciudad Guayana, Mayo del 2026.

ÍNDICE

INTRODUCCION	3
TOKENIZACION	4
IMPLICACION CON LOS COMPILADORES	5
ANÁLISIS LÉXICO.....	5
EL PROCEDIMIENTO DE TOKENIZACIÓN.....	5
IDENTIFICACIÓN DE LEXEMAS.....	6
CREACIÓN DEL TOKEN	6
INTERACCIÓN CON LA TABLA DE SÍMBOLOS.....	6
IMPLICACION EN LA ESTRUCTURA DEL COMPILADOR	7
CONEXIÓN MEDIANTE EL ANÁLISIS SINTÁCTICO	7
GESTIÓN DE ERRORES LÉXICOS.....	7
TRANSFERIBILIDAD Y EFICIENCIA	7
NLP Y IA.....	8
EJEMPLOS ENTRE NLP Y LENGUAJE FORMAL	10
BLOQUES DE CONSTRUCCIÓN.....	10
MANEJO DE ERROR DE SINTAXIS.....	11
CONCLUSIÓN	13
BIBLIOGRAFIA	14

INTRODUCCION

El lenguaje es el puente entre la intención humana y la ejecución computacional pero antes de que una máquina pueda interpretar una instrucción, ensamblar un programa o dar respuesta a una pregunta conversacional, primero debe procesar la entrada de texto y transformarla en unidades lógicas y a este proceso se le conoce como tokenización.

El objetivo de este documento es abordar y contrastar las diferencias conceptuales de la tokenización en ambos mundos, a través del análisis de su implicación directa en la arquitectura de los compiladores y el desglose de ejemplos prácticos que demuestran cómo este proceso ha sido adaptado para cumplir a dos propósitos estructuralmente opuestos, el determinismo innegociable de la máquina y la flexibilidad predictiva de la inteligencia artificial.

TOKENIZACION

La tokenización consiste en el proceso de convertir una secuencia de texto en unidades individuales o tokens, estos tokens representan las partes más pequeñas del texto que tienen sentido para la tarea que se está realizando, la tokenización suele ser primer paso en el proceso de preprocesamiento de texto en el PLN.

Dentro de los lenguajes formales de programación, la tokenización es determinista y se refiere a la primera fase del proceso de compilación, donde los tokens son palabras reservadas, identificadores, constantes, operadores o símbolos de puntuación basados en las reglas del lenguaje, que la máquina necesita para poder ejecutar una orden sin margen de error, dentro de los cuales participan el lexema, quien representa la secuencia de caracteres concreta que con un patrón, el patrón a su vez se refiere a la regla que define el conjunto de lexemas que pueden representar componente léxico o token mismo.

Por otro lado, dentro del PLN, la tokenización permite preparar datos de texto para tareas analíticas y computacionales, los tokens pueden ser palabras, subpalabras o caracteres, dependiendo de las necesidades específicas de la tarea, su importancia radica en su capacidad simplificar el análisis de texto, facilitar la extracción de características que permiten realizar diversas tareas de PLN como clasificación de textos, análisis de opiniones y reconocimiento de entidades nombradas, así como también ayuda estandarizar la entrada de texto, logrando que sea más fácil de procesar para los algoritmos.

IMPLICACION CON LOS COMPILADORES

ANÁLISIS LÉXICO

Este constituye la etapa inicial de un compilador y su objetivo consiste en interpretar el flujo de caracteres del código fuente y organizarlos en unidades lógicas denominadas lexemas, que posteriormente son clasificados como tokens.

En esta fase, el analizador léxico funciona como un filtro que se encarga de los siguientes procesos:

- **Eliminación de ruido:** Elimina espacios vacíos, tabulaciones y saltos de línea que no alteran la semántica en la mayoría de los lenguajes.
- **Depuración de comentarios:** Suprime los comentarios del desarrollador, ya que no son necesarios para la ejecución.
- **Detección de patrones:** Emplea Expresiones Regulares para identificar si una secuencia de texto es un número, una palabra clave (if, while, return) o un identificador de variable.

EL PROCEDIMIENTO DE TOKENIZACIÓN

Este procedimiento sigue una serie lógica de cambios que posibilita al compilador interpretar la sintaxis del código y el procedimiento consta de las siguientes etapas:

1. IDENTIFICACIÓN DE LEXEMAS

Donde un lexema es la cadena de caracteres exacta en el código. Por ejemplo, en la expresión `suma = 10 + 5;`, los lexemas son `suma`, `=`, `10`, `+`, `5` y `;`.

2. CREACIÓN DEL TOKEN

Donde para cada lexema, el analizador produce un par que comúnmente tiene la siguiente estructura:

- **Nombre del Token:** Es un signo abstracto que denota la clase gramatical, por ejemplo. IDENTIFICADOR, OPERADOR_ASIG, NÚMERO.
- **Valor del Atributo:** Es un apuntador o registro en la Tabla de Símbolos donde se almacena la información particular del lexema (denominación de la variable, valor de la constante y el tipo de dato).

3. INTERACCIÓN CON LA TABLA DE SÍMBOLOS

Cada vez que se crea un token de tipo identificador el analizador léxico lo anota en la tabla de símbolos, si el identificador ya está presente, solo devuelve el puntero a esa entrada para que el compilador entienda que se está hablando de la misma entidad.

IMPLICACION EN LA ESTRUCTURA DEL COMPILADOR

Los tokens no solo ayudan a depurar el código, su presencia determina la eficacia y factibilidad de las fases siguientes:

1. CONEXIÓN MEDIANTE EL ANÁLISIS SINTÁCTICO

El analizador sintáctico no recibe caracteres, recibe una secuencia de tokens y objetivo es el de comprobar que la secuencia de esos tokens cumpla con las normas de la gramática del idioma, como por ejemplo, que tras un "if" aparezca un paréntesis de apertura.

2. GESTIÓN DE ERRORES LÉXICOS

La utilización de los tokens facilita la detección temprana de errores de "ortografía", ya que si el programador introduce un símbolo no válido en el lenguaje, el compilador genera un error léxico y detiene el proceso para no desperdiciar recursos en análisis posteriores.

3. TRANSFERIBILIDAD Y EFICIENCIA

Al transformar el código fuente en tokens que generalmente se representan internamente como enteros o enumeraciones, el compilador disminuye significativamente el uso de memoria pues el comparar un entero es mucho más veloz para el procesador que comparar textos repetidamente a lo largo del proceso de compilación.

NLP Y IA

El procesamiento del lenguaje natural o NPL es el campo de la inteligencia artificial que permite que las máquinas puedan entender, procesar y responder al lenguaje humano, y es esencial en IA porque sin esta capacidad no existirían los asistentes virtuales, los chatbots, los traductores ni los modelos de texto que usamos hoy diario.

El procesamiento de lenguaje natural (PNL), o más conocido como NPL por sus siglas en inglés, permite a los ordenadores y dispositivos digitales reconocer , comprender y generar texto y voz combinando la lingüística computacional utilizando el modelado del lenguaje humano basado en reglas junto con el modelado estadístico, el machine learning (aprendizaje automático) y el deep learning (aprendizaje profundo).

Este tiene como función tomar enormes volúmenes de texto, aprende significados, relaciones y patrones entre palabras y después aplica ese conocimiento para la interpretación de frases nuevas. Aplica métodos como la tokenización, embeddings, entre otras más. La importancia de esta en la IA es fundamental ya que transforma el lenguaje de los humanos en algo que una máquina tiene la posibilidad de procesar, teniendo en cuenta todos los matices, esto hace que la IA no dependa de órdenes estrictas y pueda interactuar con nosotros de manera más natural.

Al trabajar con el procesamiento del lenguaje natural (PNL), un desafío recurrente es lidiar con las palabras fuera del vocabulario (OOV). Se trata de palabras que un

modelo nunca ha encontrado durante el entrenamiento, pero aparecen en el uso cotidiano. Dado que el modelo no tiene una representación aprendida para la palabra, no puede interpretarla ni procesarla directamente.

Para solucionar este problema la IA moderna utiliza la tokenización por subpalabras, esta consiste en dividir el texto en unidades mayores que un solo carácter pero menores que una palabra completa. Esto resulta útil al trabajar con palabras poco frecuentes.

La tokenización de subpalabras ayuda a manejar palabras fuera del vocabulario en tareas de PNL y para idiomas que forman palabras combinando unidades más pequeñas.

Algunos de los desafíos en la tokenización es la ambigüedad ya que las palabras pueden contener varios significados y las reglas de tokenización no siempre capturan el significado deseado correctamente. Palabras como “San Francisco” plantean el dilema de si tratarlas como tokens individuales o dividir las.

EJEMPLOS ENTRE NLP Y LENGUAJE FORMAL

BLOQUES DE CONSTRUCCIÓN

En los lenguajes formales, los bloques son finitos y rígidamente definidos. El analizador léxico busca lexemas exactos que correspondan a una gramática preestablecida, por el contrario, en el PLN los bloques son fluidos y dependen de un vocabulario que evoluciona.

Característica	Lenguaje Formal (Compiladores)	Procesamiento de Lenguaje Natural (NLP)
Unidad Mínima	Tokens predefinidos (palabras reservadas, operadores, símbolos).	Palabras, sub-palabras o caracteres según la tarea.
Identificación	Se usan Expresiones Regulares para clasificar tipos como número o identificador.	Se usan Embeddings y modelos estadísticos para aprender relaciones y patrones.
Referencia	Se anotan en una Tabla de Símbolos para mantener la unicidad de las variables.	Se basan en un vocabulario de entrenamiento; lo desconocido se trata como OOV

MANEJO DE ERROR DE SINTAXIS

La principal diferencia aquí radica en la tolerancia. Un compilador es un sistema binario: el código es válido o genera un error que detiene el proceso. El NLP busca la interpretación más probable a pesar de la imperfección.

Dentro de los compiladores si el analizador sintáctico recibe una secuencia de tokens que no cumple la gramática se genera un error fatal. En cambio el NLP Ante palabras mal escritas o desconocidas, se emplea la tokenización por subpalabras, lo que permite que el sistema descomponga una palabra extraña en unidades menores que sí conoce, manteniendo la fluidez de la interacción natural sin detener el análisis.

SEPARACIÓN Y CONCATENACIÓN

El tratamiento del ruido y la unión de conceptos varían drásticamente entre ambos campos siendo estos la eliminación frente a conservación de espacios y la concatenación semántica.

La Eliminación contra la Conservación de Espacios, donde el compilador funciona como un filtro que elimina espacios en blanco, tabulaciones y comentarios, ya que no aportan semántica a la ejecución. En el NLP, la separación no siempre es clara; por ejemplo, el espacio en "San Francisco" plantea el dilema de si tratarlo como un solo token o dos.

En la concatenación semántica, dentro de los lenguajes formales esta suele ser una operación explícita, como sumar dos cadenas por ejemplo. En cambio, en

el NLP, la concatenación de tokens busca reconstruir el contexto y los matices del lenguaje humano para que la máquina entienda y no solo reciba órdenes estrictas.

CONCLUSIÓN

Finalmente, la tokenización demuestra que el primer paso para que una máquina procese cualquier entrada de texto dicta por completo la naturaleza del sistema. Aunque el concepto base es idéntico, su aplicación práctica revela una profunda diferencia entre cómo construimos software tradicionalmente y cómo intentamos replicar la comprensión humana.

Dominar las diferencias entre la tokenización formal y la tokenización del PLN va más allá de un ejercicio teórico; es una necesidad para comprender la evolución computacional en una era donde la Inteligencia Artificial comienza a generar, evaluar e interactuar con código fuente clásico por lo que entender cómo leen las máquinas nos resulta fundamental para diseñar las herramientas y los compiladores del futuro.

BIBLIOGRAFIA

GeeksforGeeks. (23 de julio, 2025). *What is tokenization in Natural Language Processing (NLP)?* <https://www.geeksforgeeks.org/nlp/tokenization-in-natural-language-processing-nlp/>

GeeksforGeeks. (6 de mayo, 2026). *Tokenization in NLP.* <https://www.geeksforgeeks.org/nlp/nlp-how-tokenizing-text-sentence-words-works/>

GeeksforGeeks. (11 de abril, 2026). *Phases of a compiler.* <https://www.geeksforgeeks.org/compiler-design/phases-of-a-compiler/>

CompiladoresUGB. (1 de junio, 2016). *Concepto de token, patrones, lexema y atributo.* <https://compiladoresugb.wordpress.com/2016/06/01/concepto-de-token-patrones-lexema-y-atributo/>

Redacción UNIR. (12 de diciembre, 2025). *¿Qué es el Procesamiento del Lenguaje Natural (NPL) y por qué es esencial en Inteligencia Artificial?* Universidad Virtual | UNIR PE. <https://peru.unir.net/revista/ingenieria/que-es-npl/>

Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2008). *Compiladores: Principios, Técnicas y Herramientas.* Pearson Educación.

Appel, A. W. (2004). *Modern Compiler Implementation in Java/C/ML.* Cambridge University Press.

Louden, K. C. (2004). *Construcción de Compiladores: Principios y Práctica.* Thomson Learning.

Hopcroft, J. E., Motwani, R., & Ullman, J. D. (2007). *Introducción a la Teoría de Automatas, Lenguajes y Computación.*